Auditable Rust/CUDA Systems for Parameter-Constrained Language Model Training

Cedric Haddad
Department of Computer Science
University of Chicago
Chicago, IL 60637

Abstract

OpenAI's Parameter Golf challenge asks participants to train the best language model that fits in a 16 MB artifact and trains in under 10 minutes on 8×H100 GPUs, evaluated by tokenizer-agnostic bits-per-byte (BPB) compression on FineWeb validation data. This setting makes implementation details—precision flow, memory traffic, data residency, CUDA graph capture, optimizer communication, evaluation-time adaptation, and artifact accounting—part of the modeling problem. We present a Rust/CUDA training stack for this challenge and report the current end-to-end train-step profiling evidence from our latest frontier-derived target. Compared with our first valid Rust/CUDA record-path implementation, which exposed a 91.2 second step time on an earlier heavier record shape, the current exact-gradient #2135-shaped profile runs at 127.11 ms/step, a roughly 718× reduction in step time. More conservatively, on the current #2135-shaped target, the optimized graph profile is 1.11× faster than a graph-disabled stage-timing probe (127.11 ms/step vs. 141.53 ms/step), while a diagnostic no-loss profile reached 116.96 ms/step. The current implementation uses typed runtime specs, BF16 direct-compact backward execution, a GPU token-ring schedule with zero hot-path host batch flattening, no audited F32/BF16 bridge launches, no-loss CUDA graph capture, chunked BF16 output cross-entropy, and sharded Parallel Muon graph paths. The fastest quality-bearing diagnostic profiles reached 2.254799 BPB before export and 2.438243 BPB after export, far from the public frontier but useful because they exposed the remaining gap between systems throughput and model/export quality. We do not claim a final leaderboard BPB: no compliant full train/eval/export run has completed, the final artifact budget is still unproven, and 3-seed full-validation statistics are absent. The paper's contribution is an audit-first systems account of the gap between a from-scratch Rust/CUDA training substrate and a record-valid Parameter Golf submission, including detailed profiling results, what the project taught us about constrained training systems, and the remaining blockers to matching the public 1.05651 BPB stack.

1 Introduction

Most language-model training research is built on Python frameworks such as PyTorch and JAX. For many settings, this is the correct engineering tradeoff: automatic differentiation, mature kernels, dynamic experimentation, and a large ecosystem matter more than absolute control over every kernel boundary. Parameter Golf is different. It is a challenge to train the best language model that fits in a 16 MB artifact and trains in under 10 minutes on 8×H100 GPUs, scored by bits per byte (BPB) on the FineWeb validation set [1]. The official artifact budget counts code bytes plus compressed model bytes, with a decimal cap of 16,000,000 total bytes [2]. Evaluation must also run under the 10-minute cap, and score-first test-time training is allowed only on validation tokens that have already been scored [3].

These rules make Parameter Golf a full-stack systems problem. A model that saves 10 MB but needs too much evaluation-time adaptation does not fit the envelope. A training loop that wastes 20 ms per step may lose hundreds of optimizer updates inside the 600-second budget. A tokenizer or byte-counting error can produce an invalid BPB. A compressed model that fits in 16 MB can still fail if the code is not counted. Even a "fast" proxy benchmark can be misleading if it uses the wrong context length, token budget, recurrence schedule, or loss path.

This project began from the hypothesis that a Rust/CUDA implementation could make these tradeoffs explicit enough to audit and optimize. The original proposal emphasized speculative Rust-enabled research directions: a quantization-scheme compiler via procedural macros, custom fusions beyond `torch.compile`, and compile-time architecture specialization. The current system does not yet complete those speculative components. Instead, it first attacks a prerequisite: reconstruct a current-frontier-shaped training surface in Rust/CUDA with enough auditability that performance numbers mean what they claim to mean.

The feedback on an earlier draft asked for two presentation changes: (1) highlight speedup in the abstract and introduction, and (2) add an overview figure. We address both here. Figure 1 summarizes the systems problem, and Tables 2–4 collect the profiling evidence we currently have. The key speed result is that our latest exact-gradient #2135-shaped profile reaches 127.11 ms/step. This is already in the same order of magnitude as the public frontier's roughly 120–130 ms/step region, but it is not a record-valid result because it has not yet been embedded in a complete train/eval/export run with a final artifact and official BPB.

The quality results tell the complementary story. Our best diagnostic profiles reached roughly 2.25–2.44 BPB, while the public target is near 1.0565 BPB. This gap is large, but it is scientifically useful: it shows that the project succeeded first at building a fast enough Rust/CUDA instrument to expose the next bottleneck, not at producing the final compressed model.

2 Background: Parameter Golf and the Moving Frontier

BPB. BPB stands for bits per byte. It is a compression-style language-modeling score: lower BPB means the model needs fewer bits on average to describe each byte of validation text. Formally, if a model assigns negative log-likelihood L nats to a validation byte stream of length B bytes, then

$$\mathrm{BPB} = \frac{L}{B \log 2}.$$

A lower BPB indicates better next-byte predictive performance normalized by text bytes rather than tokenizer-dependent token counts.

Public frontier. The public leaderboard moved quickly during the project. PR #1855 reported a 3-seed mean of 1.06108 BPB with an 11-layer, 512-dimensional, 8-query-head/4-KV-head transformer using U-Net skips, parallel residuals, Polar-Express Newton-Schulz Muon, LQER asymmetric rank-4 correction, SparseAttnGate, BOS-fixed SmearGate, fused softcapped CE, int6/int7 quantization, per-group compression, and phased TTT [5]. Later, PR #2135 reported a 1.05651 BPB 3-seed mean using a PR #2130-derived stack with GPTQ calibration batches increased to 32, with maximum artifact size 15,947,490 bytes and maximum evaluation time 567.1 seconds [6]. The repository README now lists 1.0565 as the leaderboard score for the Calib32 token-only n-gram + AsymLogit stack [4].

This frontier drift changed the target of our Rust implementation. Early versions targeted a 2048-context, 786,432-token global batch shape inherited from an earlier understanding of the frontier. The current active target is the #2135 audit shape: 1024 training context, 2560 evaluation/TTT context, 524,288 global train tokens per step, and 8 GPUs. The local batch arithmetic is:

$$\text{local microbatches/rank} = \frac{524{,}288}{8 \times 1024} = 64.$$

This correction matters: timing a different context length or batch shape can easily dominate any kernel-level comparison.

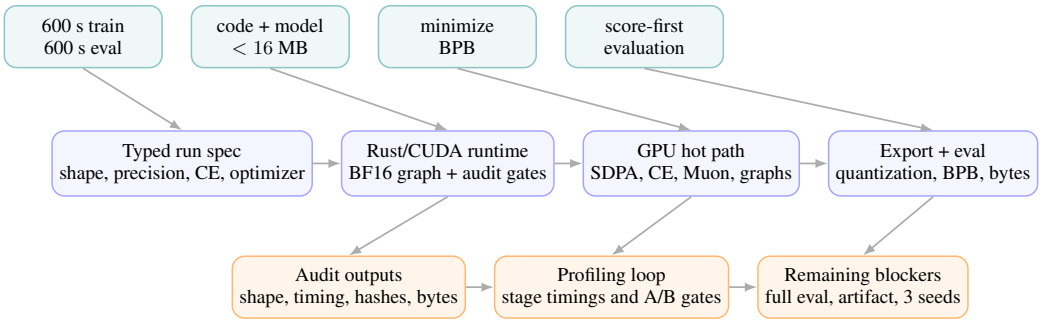

Figure 1: Overview. Parameter Golf couples four constraints—time, bytes, BPB, and score-first evaluation. Our Rust/CUDA stack makes those constraints explicit through typed specs, an auditable GPU runtime, measured kernel paths, and artifact/evaluation checks.

3 Rust/CUDA System Design

3.1 Workspace organization

The implementation is a Rust workspace divided into functional crates: `pg-core` for GPU tensor and CUDA/NCCL utilities; `pg-kernels` for CUDA/cuDNN/cuBLAS-facing kernels; `pg-model` for model configuration, explicit forward/backward state, and runtime profiles; `pg-optim` for AdamW and Muon variants; `pg-data` for token and byte-sidecar loading; `pg-quant` for artifact export, quantization metadata, and compression; `pg-eval` for sliding evaluation and GPU LoRA/phased TTT; and `pg-train` for record-shaped training, timing, and audit output.

The most important architectural choice is the spec layer. Runtime-critical choices are represented as typed fields rather than as untracked shell variables: attention backend, compute precision, output CE backend, distributed optimizer backend, evaluation adaptation backend, backward-chain profile, CUDA graph profile, NCCL overlap mode, record profile, token schedule, and quantization/export settings. This reduces the chance that a run labeled "record-shaped" is actually using a debug kernel, synthetic data path, wrong context length, full-logit fallback, or CPU batch pipeline.

3.2 Current active target

The active branch described in the latest report is `finish-frontier-2135`. The current primary audit spec is `frontier_2135_audit_target.toml`. It targets:

Component	Current audit target
Architecture family	PR #2135-derived, `frontier_1855_like` lineage
Training context	1024 tokens
Evaluation/TTT context	2560 tokens
Global train tokens/step	524,288
World size	8 H100 GPUs
Local microbatches/rank	64
Data requirement	canonical CaseOps train/validation split
Precision profile	BF16 direct-compact backward chain
Data schedule	GPU token-ring full schedule
Graph profile	no-loss backward CUDA graph, not full train-step graph
Optimizer	sharded Parallel Muon graph paths
NCCL overlap	instrumented, not promoted

The latest pre-train audit confirms canonical data properties: `canonical_caseops_dataset=true`, 80 train shards, 50,000 validation documents, 47,851,520 validation tokens, matching sidecar tokens, and a train/validation non-overlap proof. A critical correction is that the #2135 scoring target uses the 2560-context cropped target count, not the raw stored validation-token length.

3.3 Current GPU hot path

The current optimized path includes:

- BF16 direct-compact backward chain, including BF16 SDPA backward tail, direct QKV tail packing, BF16 QKV dX output, compact SparseAttnGate gradient input, and direct compact QKV norm/residual reduction.
- SparseAttnGate/XSA-adjacent fusion through BF16 BHSD paths, compact gradient-input variants, and warp-head backward variants. This is not true XSA-inside-SDPA.
- Exact recurrent boundary fusion, including a pass-2 QKV/norm/residual tail that precomputes the pass-1 MLP residual boundary so pass 1 can skip its initial MLP residual elementwise stage.
- Chunked BF16 output CE as the best practical default. Output CE is no longer the main measured bottleneck, and the exact fused CE path is present but not the decisive current limiter.
- Sharded Parallel Muon local/pre-norm graph paths, which reduce exposed bank-update wall time but do not by themselves close the speed gap.
- GPU token-ring sampler evidence: optimized proxy paths report zero host batch flatten calls and zero H2D batch bytes.

4 Methodology

4.1 Audit-first profiling

We report only profiling results with explicit shape and runtime context. Several results are non-comparable by design: early profiles used a different frontier shape, and diagnostic profiles can be faster while not carrying exact-gradient record evidence. We keep those results because they explain the engineering trajectory, but we label them accordingly.

4.2 Validity levels

We distinguish three levels of evidence.

Systems-valid train-step timing. A timing is systems-valid if it uses the intended training shape, world size, backend choices, token schedule, and graph/optimizer settings. The 127.11 ms/step result is systems-valid for the current exact-gradient #2135-shaped proxy.

Record-valid run. A run is record-valid only if the full training phase completes under 600 seconds, full evaluation completes under 600 seconds, code plus compressed model fit under 16,000,000 bytes, validation BPB is computed correctly, and score-first evaluation-time adaptation uses only already-scored validation tokens.

Leaderboard-valid result. A leaderboard-valid result additionally requires sufficient seed statistics and a public, reproducible submission package. We do not claim this level.

5 Profiling Results

5.1 Public targets

Table 1 summarizes the public comparison points. PR #1855 is the merged stack that shaped much of the current Rust target. PR #2135 is the current Calib32 target the Rust branch is chasing.

Table 1: Public Parameter Golf comparison points used to define the target.

Reference	Reported BPB	Runtime / artifact facts	Relevance
PR #1855	1.06108, 3-seed mean	$\sim$121.7 ms/step; artifacts $<$16 MB; max eval 508.8 s	merged frontier lineage
PR #2135	1.05651, 3-seed mean	15.95 MB max; 600 s train/eval; max eval 567.1 s	current target shape

5.2 Development and profiling timeline

Table 2 collects the main profiling results available from the project reports. The most important number is the exact-gradient #2135-shaped graph profile at 127.109924 ms/step. The 116.963 ms/step profile is included, but we treat it as diagnostic rather than record-quality exact-gradient evidence.

Table 2: Available profiling results. Some entries use different shapes or diagnostic settings; the "Role" column states how each should be interpreted.

Profile	Shape / setting	Timing	Role
Initial full 8×H100 record-mode attempt	older 2048 context, 786,432 tokens/step	91,218.335 ms/step; 7 steps in 638.528 s	Validated that the real record path was far slower than proxy paths; not comparable to #2135 shape.
CUDA single smoke	small smoke	75.799 ms/step over 4 steps	Sanity check, not a record-shape benchmark.
CUDA single proxy	proxy	53.196 ms/step over 32 steps; proxy BPB 7.132411	Fast functional proxy; not quality evidence.
8-GPU smoke	short distributed smoke	312.661 ms/step over 4 steps	Distributed-sync sanity check.
8-GPU proxy	proxy	132.554 ms/step over 32 steps; proxy BPB 5.834740	Earlier distributed proxy, not final shape/quality evidence.
Diagnostic no-loss / fast profile	current-shape diagnostic	116.963 ms/step mean	Shows the speed region attainable, but not exact-gradient record evidence.
Exact #2135 graph proxy	1024 context, 524,288 tokens/step	127.109924 ms/step	Current primary systems evidence.
Chunked recurrent scale-fusion candidate	same as exact proxy	127.131505 ms/step	Neutral/slightly worse; not promoted.
Graph-disabled stage probe	same target, graph disabled	141.531 ms/step	Used for stage attribution and graph benefit estimate.

The most visually dramatic speedup is from the initial valid old-shape path to the current exact #2135 profile:

$$\frac{91{,}218.335}{127.109924} \approx 717.6\times .$$

This is not an apples-to-apples benchmark because the target shape changed. It is still useful as an engineering progress marker: the system moved from an infeasible record path to a current-frontier-shaped exact-gradient profile near the target runtime regime.

A more conservative same-target comparison is the graph-disabled stage probe versus the optimized graph profile:

$$\frac{141.531}{127.109924} \approx 1.11\times .$$

This measures the combined benefit of graph/profile choices under the current target more directly, although it is still not a full train/eval/export run.

5.3 Exact #2135 recurrence profile

Table 3 shows the recurrence-specific timing reported for the exact #2135-shaped graph proxy and the scale-fusion candidate.

The fusion candidate is essentially neutral: it slightly improves active recurrence timing while slightly worsening inactive recurrence timing and overall step time. It should not be promoted without a

Table 3: Exact #2135-shaped recurrence timing. Active recurrent replay is the decisive remaining speed blocker.

Profile	Overall ms/step	Active recurrence ms/step	Inactive recurrence ms/step
Exact graph short proxy	127.109924	155.655481	119.426539
Chunked scale-fusion candidate	127.131505	155.576088	119.475299

stronger measured gain. The broader conclusion is that active recurrent replay, not output CE, is currently the decisive speed blocker. To hit a strict $\leq$120 ms/step median target, the system needs roughly 8–17 ms/step additional improvement on the full exact profile, likely from reducing the active recurrent steps by 25–30 ms.

5.4 Stage timing

The graph-disabled stage-timing probe reported 141.531 ms/step. Its main components are shown in Table 4. These timings are useful because they identify where speed work should focus next.

Table 4: Graph-disabled stage-timing probe at 141.531 ms/step.

Stage	ms/step
Forward	44.1
MLP backward	20.4
QKV backward	22.0
Attention SDPA backward	14.2
Gate/XSA-adjacent work	6.9
Bank update	13.1
Measured total	141.531

The table supports the current engineering diagnosis: further gains should target recurrent replay and backward compute, not CE first. Muon NS step reductions, sharded Muon graphing, fused global clip, NCCL overlap, graph-side GEMM capture, and chunked compact recurrence-scale fusion were all tried. Of those, sharded Muon graphing was useful, while fused global clip regressed, NCCL overlap did not prove a clean win, graph-side GEMM capture did not improve the exact profile, and the chunked recurrence-scale fusion was neutral.

5.5 Evaluation and BPB diagnostics

The BPB evidence is diagnostic rather than record-valid. Table 5 collects the quality-bearing results that were available at the end of the project. The early proxy artifacts were intentionally small and low quality; they were useful mainly for validating artifact/eval plumbing and for exposing a LoRA shape bug. Later diagnostic runs were more informative because they came from faster profiles that could train enough to produce nontrivial language-modeling signal, but they remain far from the public frontier.

The main conclusion from these diagnostics is that step time and model quality must now be optimized together. The Rust runtime reached a plausible step-time regime, but the exported model still needs the same compression fidelity, calibration behavior, and evaluation-time adaptation quality as the winning public stack. The regression from 2.254799 BPB before export to 2.438243 BPB after export is especially important: the 16 MB artifact is not just a container for a trained model; it is part of the algorithm.

Table 5: Diagnostic BPB results available from project reports. Lower is better. None of these are claimed as record-valid full-validation scores.

Run / evaluation setting	BPB	Interpretation
CUDA single proxy	7.132411	Functional smoke-quality signal; useful mainly to verify the runtime pipeline.
8-GPU proxy	5.834740	Distributed proxy score; not final-shape quality evidence.
Early record-artifact eval on 16,384 tokens	4.548247	First artifact-eval signal on a tiny validation slice.
Fast Hybrid-ST diagnostic profile	2.254799	Best reported pre-export diagnostic quality; fast enough to diagnose model quality, but not close to the frontier.
Exported All-ST diagnostic artifact	2.438243	Post-export quality was worse, indicating that export and quantization fidelity remained central blockers.
Public #2135 target	1.05651	Current public frontier target used as the comparison point.

6 Analysis

6.1 Why the stack became fast enough to diagnose the real problem

The early implementation spent its time on the wrong computational surfaces: small proxy runs, accidental batch semantics, F32-heavy activations, host batch construction, and non-record-shaped timing. The current speedup came from changing the measured object.

First, the benchmark was corrected to the active frontier target: 1024 context and 524,288 tokens/step. Second, the runtime moved many choices into typed spec fields, making it possible to audit whether the active run truly used the intended precision, graph, recurrence, and optimizer profile. Third, BF16 direct-compact execution removed the audited F32/BF16 bridge launches in the optimized path. Fourth, the GPU token-ring schedule removed host batch flattening and hot-path H2D batch copies. Fifth, no-loss backward graph capture reduced launch overhead and stabilized the record-shaped profile. Finally, sharded Parallel Muon graph paths reduced exposed bank-update wall time.

This is an important result for the original research question. Rust did not help simply because Rust is faster than Python. It helped because the runtime could turn hidden assumptions into explicit, typed, testable state.

6.2 Why the project is still not record-ready

The project is not done for two reasons.

Quality is not yet competitive. The optimized exact profile has no full-validation BPB, and the diagnostic BPB values we do have are far from the public target: 2.254799 BPB for the best reported hybrid sparse/TTT diagnostic run and 2.438243 BPB after export for the all-sparse/TTT artifact. The public target is 1.05651 BPB. Until a full train/eval/export run produces a post-export, full-validation BPB, we cannot know whether the Rust stack has preserved the quality of the frontier algorithm.

The last speed blocker is active recurrent replay. Exact active recurrent replay remains around 155–156 ms/active step. Inactive recurrence is around 119.4 ms/step. This gap indicates that the core target is no longer basic CUDA feasibility; it is reducing the active recurrent computation without corrupting exact-gradient evidence.

6.3 What has been tried and not promoted

Negative results are useful here. Reducing Muon Newton-Schulz steps to 2 did not materially fix bank wall time. Sharded Muon local/pre-norm graphing helped but was insufficient alone. Fused global

clipping regressed and remains disabled. NCCL overlap has instrumentation and event-window proof, but it did not produce a clean timing win and is not promoted. Graph-side GEMM capture did not improve the exact profile. Chunked compact QKV/norm/residual recurrence-scale fusion was neutral or slightly worse. Several recurrence-light and score-training probes reached attractive timing or partial BPB numbers, but they are not substitutes for exact-gradient record evidence.

These outcomes narrow the remaining search: do not chase CE or bank-update first unless new profiling contradicts the current evidence. Focus on exact active recurrent replay, artifact/BPB closure, and a final full-validation run.

7 Relation to the Original Proposal

The original proposal argued for three main Rust-enabled research directions: (1) a procedural-macro quantization-scheme compiler, (2) custom fusions beyond `torch.compile`, including XSA-in-attention and BigramHash-embedding fusion, and (3) compile-time architecture specialization. The current codebase has not finished those pieces.

However, several adjacent ideas have landed:

- BF16 direct-compact backward and QKV/RoPE/gain tail fusion address the same memory-traffic problem that motivated the proposal's fused RMSNorm/QK/RoPE/q-gain direction.
- SparseAttnGate/XSA-adjacent BF16 paths are implemented, but true XSA-inside-SDPA is not.
- BigramHash fused backward support and audit surface exist, but BigramHash is not active in the #2135 target.
- Quant layout and proc-macro audit plumbing exist, but generated pack/dequant specializations and final budget proof remain unvalidated.

Thus, the paper's final claim should be narrower than the proposal. The demonstrated contribution is not that the novel fusions already beat the leaderboard. It is that the Rust/CUDA stack has become a credible experimental instrument for testing those fusions once a full record-valid baseline exists.

8 Remaining Work

A complete follow-up result must close three loops at the same time: speed, quality, and artifact validity. The speed target is a median step time near the public frontier, ideally at or below 120 ms/step on the exact #2135-shaped profile with p90 latency preserved. The quality target is a full-validation post-export BPB below the 1.05650768 public target, measured on the canonical CaseOps data path rather than a short proxy. The artifact target is a final code-plus-model bundle below 16,000,000 decimal bytes, with the model reload path producing the same score that was reported.

The immediate engineering priorities are therefore focused rather than open-ended. First, exact active recurrent replay must be reduced by roughly 25–30 ms/active step without sacrificing exact-gradient evidence. Second, the export path must preserve quality after quantization and compression; the current diagnostics show that post-export BPB can regress substantially. Third, full score-first evaluation has to run under the same 600-second limit as the competition. Fourth, NCCL overlap and persistent-CTA backward fusion should be promoted only if controlled A/B runs improve both median and p90 latency. Finally, proposal-specific research ideas—true XSA-inside-SDPA, BigramHash/embedding fusion, and generated quantization kernels—should be evaluated as ablations once the Rust baseline itself is record-valid.

9 Discussion: What We Learned

The project did not end with a leaderboard submission, but it did answer the question that motivated the original proposal more concretely than a clean success story would have. We learned that, under Parameter Golf constraints, the training stack becomes part of the model. Architecture, optimizer, precision format, data movement, graph capture, export layout, compression, and evaluation-time adaptation all determine how much useful learning fits into 600 seconds and 16 MB.

The stack we arrived at. The final Rust/CUDA stack is best understood as an auditable record-runtime prototype. It has a typed specification layer for model shape, precision, CE backend, graph profile, optimizer backend, token schedule, and artifact budget. It has a GPU runtime with BF16 direct-compact execution, cuDNN-style BF16 SDPA paths, cuBLASLt GEMM paths, explicit backward state, QKV/RoPE/gain tail fusion, SparseAttnGate/XSA-adjacent BF16 variants, exact recurrent boundary fusion, chunked BF16 output CE, and sharded Parallel Muon graph paths. It also has a GPU token-ring schedule that eliminates host batch flattening in the optimized proxy path, plus audit outputs for data hashes, step timing, artifact bytes, and readiness checks. That is a substantial systems substrate even though it is not yet a completed record.

The main systems lesson. The largest early mistake was treating fast proxy runs as evidence about the record workload. Once the batch semantics and target shape were corrected, the first valid path was too slow by orders of magnitude. The path to the current 127.11 ms/step profile came from making the measured object match the intended object: correct token budget, correct context length, correct world size, BF16 hot paths, no hot-path host batch flattening, graph-oriented execution, and stage-level timing. This is the main benefit we got from Rust/CUDA: not automatic speed, but control over what was being measured.

The main modeling lesson. Speed alone does not solve Parameter Golf. The BPB diagnostics make this clear. The best sparse/TTT diagnostic result we reported was approximately 2.25 BPB, and the post-export all-sparse/TTT artifact measured approximately 2.44 BPB. Those numbers are far from the public 1.05651 BPB target. They show that the system became fast enough to expose the next problem: the exported model and evaluation pipeline did not yet preserve frontier-quality behavior. To beat the winning stack, the Rust implementation must close both sides simultaneously: train quickly enough to fit thousands of useful updates, and export/evaluate a model whose compressed representation retains the quality of the frontier recipe.

The main export lesson. The artifact pipeline is not bookkeeping; it is part of the algorithm. A model can train quickly and still lose if quantization, low-rank correction, sidecar handling, or compression changes the distribution it represents. The project reached the point where export bugs and compression fidelity mattered more than raw feasibility. That is why future work should treat artifact round-trip BPB as a first-class metric alongside step time.

Why the project remains incomplete. The immediate reason is compute access. The competition requires full train and eval evidence on $8\times$H100 hardware. By the time the Rust stack reached a plausible record-shaped step-time regime, we had run out of Modal credits needed to execute the full sequence of $8\times$H100 experiments: long train, artifact export, full evaluation, and repeated seeds. As a result, the paper can report end-to-end systems profiling and diagnostic BPB values, but not the final evidence that would be required for a leaderboard claim: a completed 600-second train, completed 600-second full validation eval, official artifact bytes, and multi-seed statistics.

How this changes the original proposal. The proposal emphasized novel kernel fusions and a generated quantization compiler. Those ideas remain worth testing, but the project showed that they need a record-valid baseline first. The right order is now clear: first, make the frontier-shaped Rust stack fully record-valid; second, reproduce the winning public BPB under the same constraints; third, evaluate proposal-specific fusions such as true XSA-inside-SDPA, BigramHash/embedding fusion, and generated quantization kernels as controlled ablations. Without that baseline, it is impossible to tell whether a novel fusion improves the system or merely hides a larger unresolved bottleneck.

10 Conclusion

We presented a Rust/CUDA systems study of Parameter Golf under a 16 MB artifact budget and 600-second train/eval limits. The current implementation has moved from an infeasible early valid record path to an exact-gradient #2135-shaped profile at 127.11 ms/step, with diagnostic profiles reaching 116.96 ms/step. This speedup came from target-shape correction, typed runtime specs, BF16 direct-compact execution, GPU token-ring data scheduling, no-loss CUDA graph capture, and sharded Parallel Muon graphing. It did not yet come from a finished quantization compiler, true XSA-inside-SDPA, or a final leaderboard-quality artifact.

The broader lesson is that systems-level languages are not automatically faster. Their advantage in this setting is that they make the training stack itself inspectable: precision flow, data residency, kernel boundaries, optimizer communication, evaluation legality, and artifact bytes become things the runtime can measure and reject. Parameter Golf compresses all of those concerns into one objective. To beat the public 1.05651 BPB stack, the next step is not another proxy benchmark; it is a full, canonical 8×H100 train/eval/export run whose timing, BPB, and bytes all close simultaneously. The project stopped short of that evidence because the required competition-scale H100 runs exhausted our available Modal credits, not because the remaining milestones are conceptually undefined.

References

[1] OpenAI. Parameter Golf: Train the smallest LM you can that fits in 16MB. `https://github.com/openai/parameter-golf`, 2026.

[2] OpenAI. Parameter Golf README, artifact-size FAQ. `https://github.com/openai/parameter-golf`, 2026.

[3] OpenAI. Parameter Golf README, evaluation and test-time training restrictions. `https://github.com/openai/parameter-golf`, 2026.

[4] OpenAI. Parameter Golf README leaderboard, Calib32 token-only n-gram + AsymLogit stack. `https://github.com/openai/parameter-golf`, 2026.

[5] Codemath3000. Record: SP8192 + LQER + Sparse Attn Gate + BOS-Fixed SmearGate + 9-Hparam Greedy Stack — val_bpb 1.06108. Pull request #1855, `https://github.com/openai/parameter-golf/pull/1855`, 2026.

[6] Codemath3000. Record candidate: PR #2130 base + GPTQ_CALIBRATION_BATCHES=32 — val_bpb 1.05651. Pull request #2135, `https://github.com/openai/parameter-golf/pull/2135`, 2026.

[7] K. Jordan. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. *Proceedings of ICLR*, 2023.

[9] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[10] NVIDIA. CUDA Graphs. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs`, 2026.

[11] NVIDIA. cuDNN Frontend scaled dot-product attention operation. `https://docs.nvidia.com/deeplearning/cudnn/frontend/latest/operations/Attention.html`, 2026.